Lab 01 – Làm quen với Git

Yêu cầu:

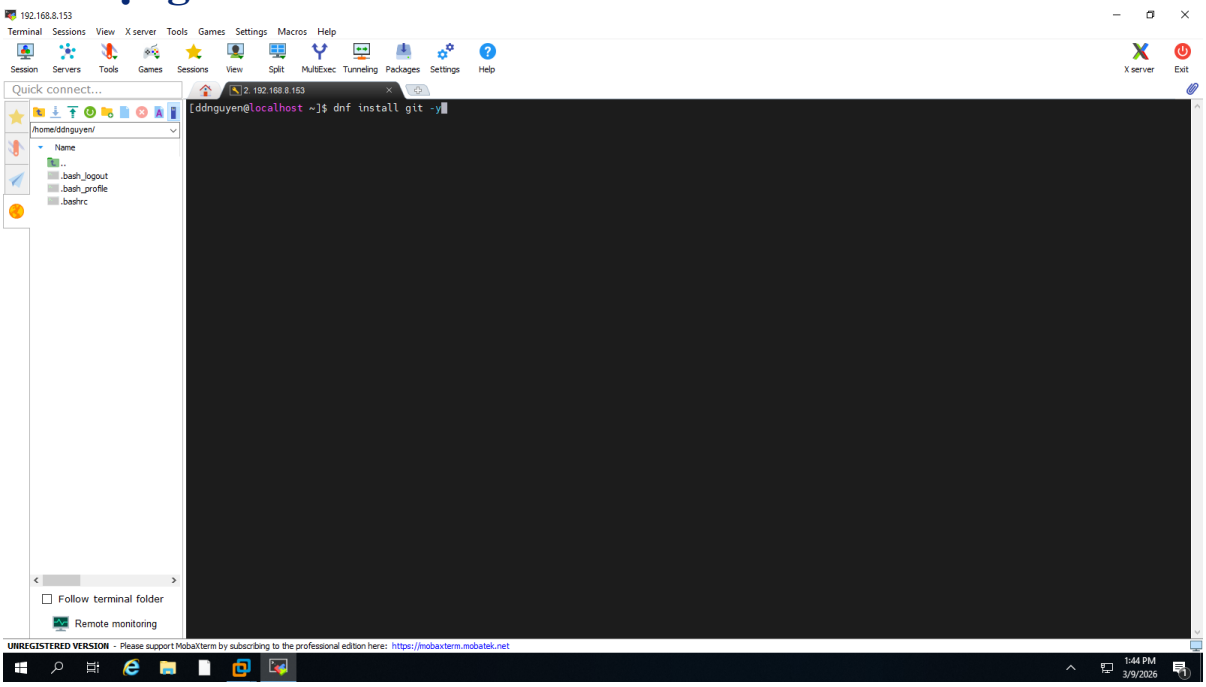
- Tạo một máy ảo Linux (Ubuntu hoặc Alma Linux).
- Clone repository sau: https://github.com/eficode-academy/git-katas.git
- Tham khảo trên Github và thực hiện các bài tập sau theo trình tự và hướng dẫn:
    - Basic Commits
    - Basic Staging
    - Investigation
    - Basic Branching
    - Fast Forward Merge
    - 3 way Merge
    - Merge Mergesort
    - Rebase Branch
    - Basic Revert
    - Reset
    - Basic Cleaning
    - Amend
    - Reorder the History
    - Advanced Rebase Interactive
    - Rebase using autosquash
    - Basic Stashing

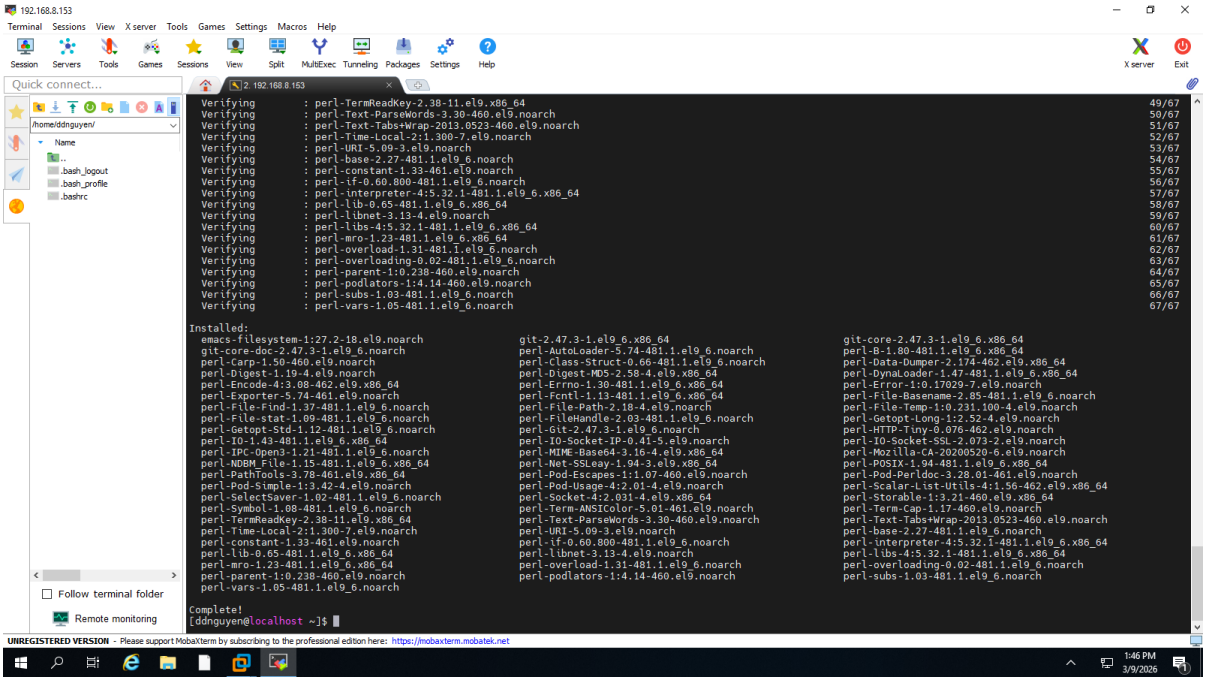
Chụp lại các bước đã thực hiện vào 1 file word (Ghi kỹ từng bài tập).
Tạo một tài khoản github, tạo một repo và commit kết quả kèm file word lên repo đó. Bài lab sẽ được kiểm tra trên repo của sinh viên.

Bài làm

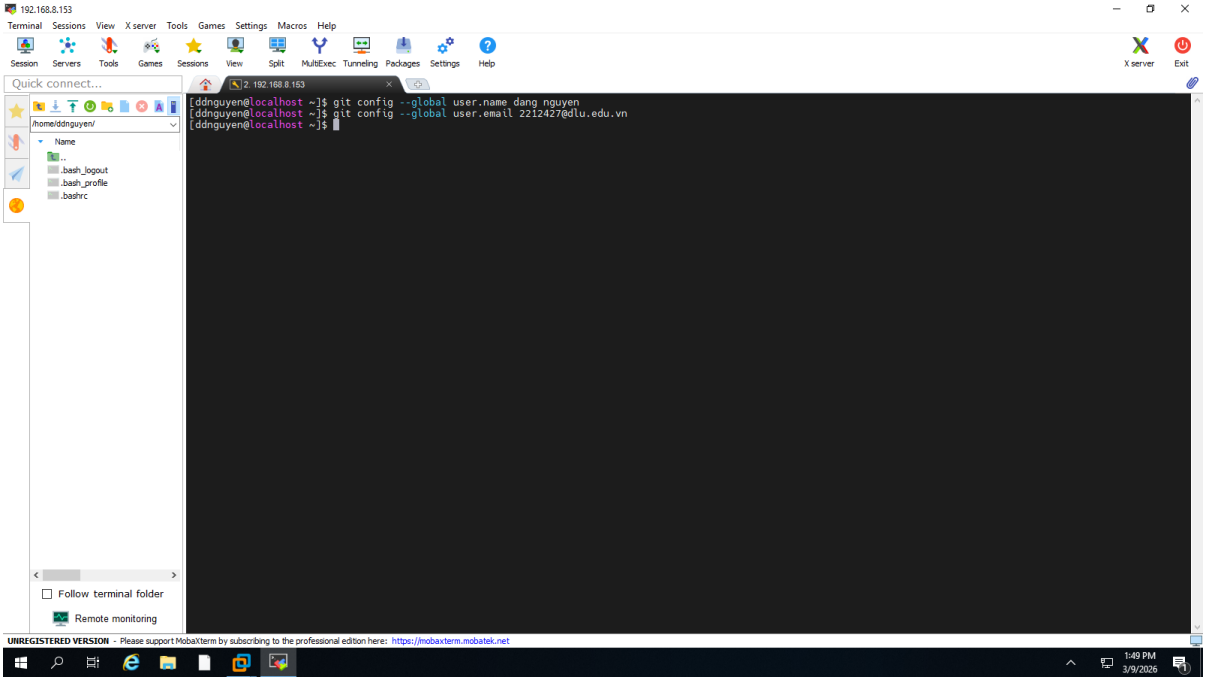
1. Chuẩn bị

Cài đặt git:

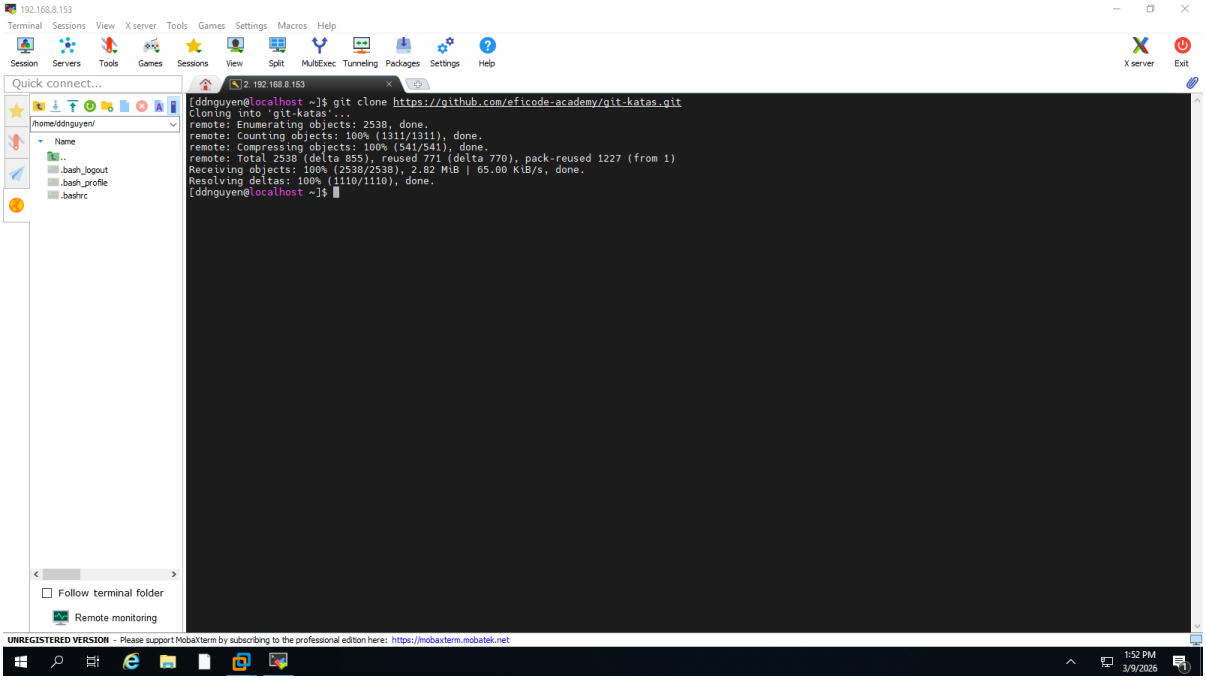




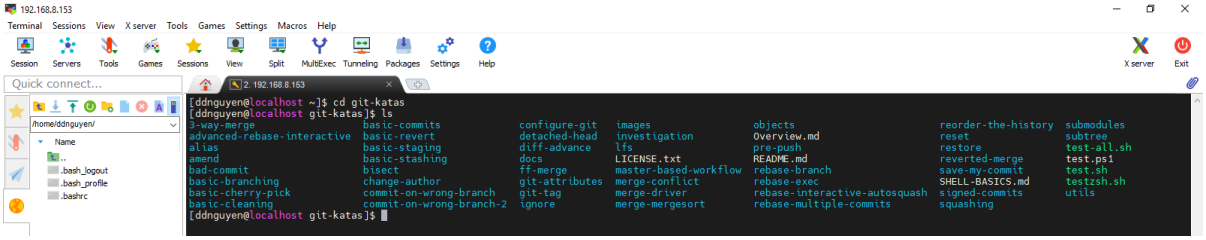
Cấu hình thông tin người dùng



Clone repository

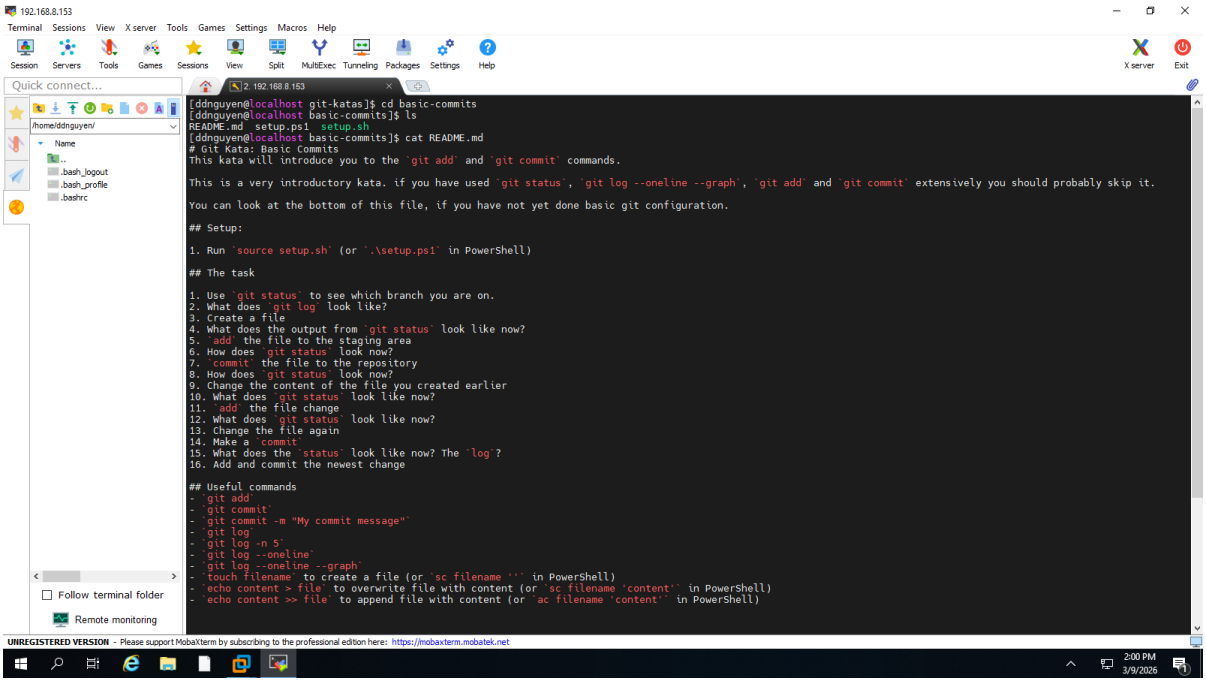


Di chuyển vào thư mục git-katas



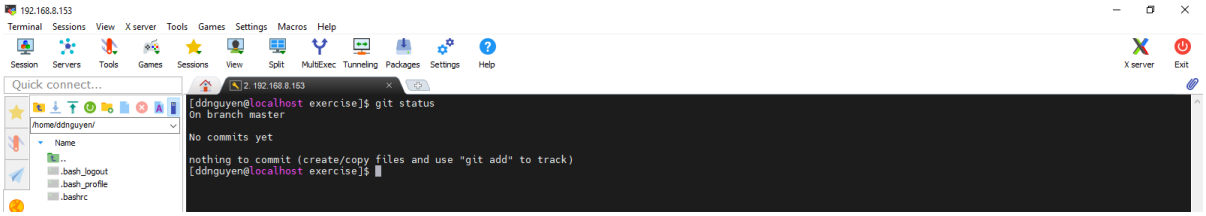
2. Thực hành

2.1 Basic commits



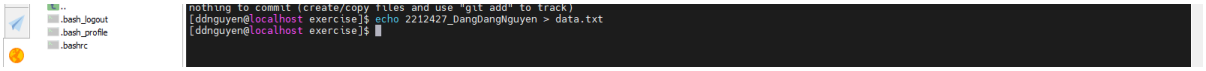
Setup:

Task 1:

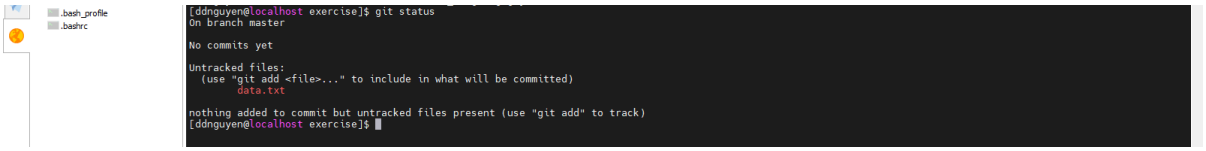


Task 2:

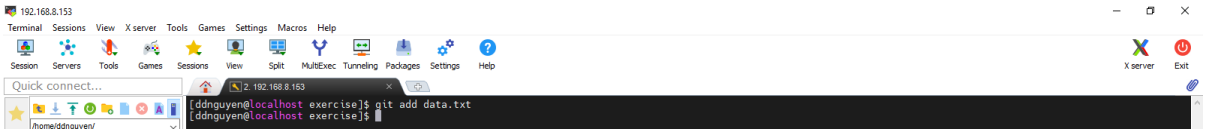
Task 3:



Task 4:



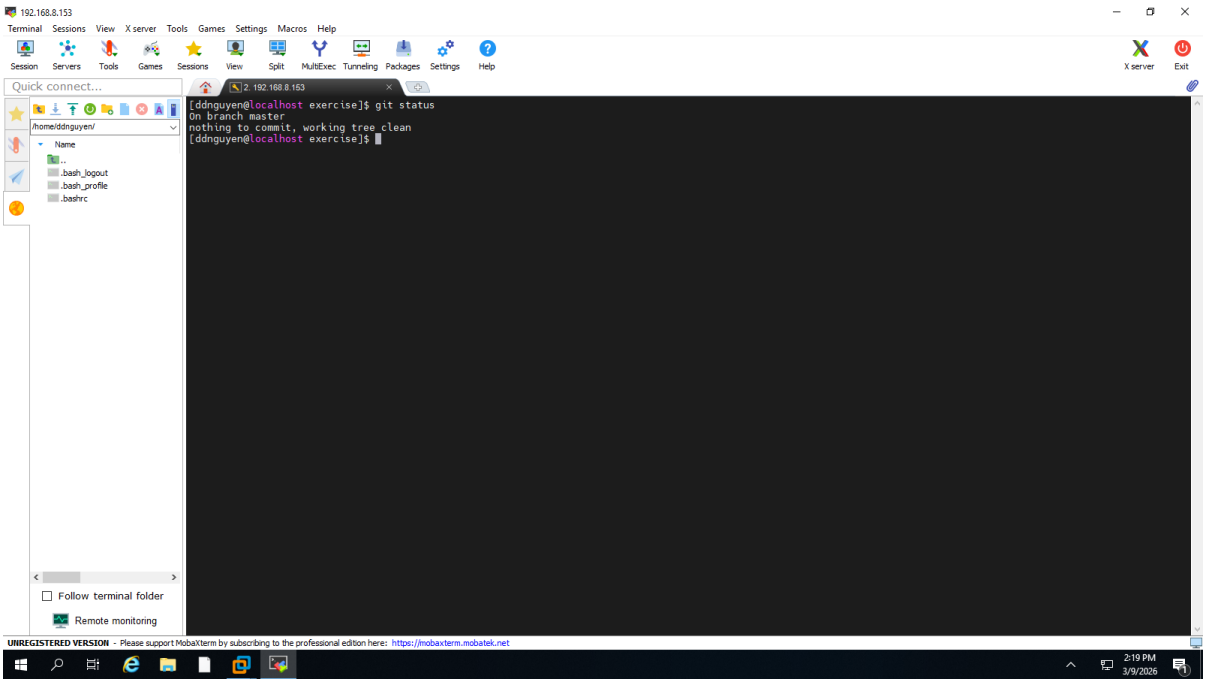
Task 5:



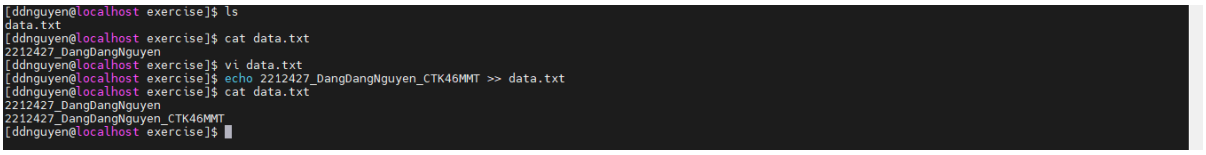
Task 6:

Task 7:

Task 8:



Task 9:



Task 10:

```
[ddnguyen@localhost exercise]$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   data.txt

no changes added to commit (use "git add" and/or "git commit -a")
[ddnguyen@localhost exercise]$
```

Task 11:

```
[ddnguyen@localhost exercise]$ git add data.txt
[ddnguyen@localhost exercise]$
```

Task 12:

```
[ddnguyen@localhost exercise]$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   data.txt

[ddnguyen@localhost exercise]$
```

Task 13:

```
[ddnguyen@localhost exercise]$ ls
data.txt
[ddnguyen@localhost exercise]$ cat data.txt
2212427_DangDangNguyen
2212427_DangDangNguyen_CTK46MMT
[ddnguyen@localhost exercise]$ sudo nano data.txt
[ddnguyen@localhost exercise]$ cat data.txt
2212427_DangDangNguyen
2212427_DangDangNguyen_CTK46MMT
2212427_DangDangNguyen_CTK46MMT_CDMMT2
```

Task 14:

```
[ddnguyen@localhost exercise]$ git commit -m "Task 14"
[master 2529160] Task 14
 1 file changed, 1 insertion(+)
[ddnguyen@localhost exercise]$
```

Task 15:

```
[ddnguyen@localhost exercise]$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   data.txt

no changes added to commit (use "git add" and/or "git commit -a")
[ddnguyen@localhost exercise]$ git log
commit 25291607fe29607296628f7b301edad552537560 (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Mon Mar 9 14:31:24 2026 +0700

    Task 14

commit 8b0c0d10e335fb19e9a11e8e5abc6225ff01d654
Author: dang <2212427@dlu.edu.vn>
Date:   Mon Mar 9 14:18:48 2026 +0700

    task 7
[ddnguyen@localhost exercise]$
```

Task 16:

```
[ddnguyen@localhost exercise]$ git add data.txt
[ddnguyen@localhost exercise]$ git commit -m "Task 16"
[master 6d9df58] Task 16
 1 file changed, 1 insertion(+)
[ddnguyen@localhost exercise]$
```

2.2 Basic staging

```
[ddnguyen@localhost git-katas]$ ls
3-way-merge                  basic-commits             configure-git   images                  objects                       reorder-the-history  submodules
advanced-rebase-interactive  basic-revert              detached-head   investigation           Overview.md                   reset                subtree
alias                        basic-staging             diff-advance    lfs                     pre-push                      restore              test-all.sh
amend                        basic-stashing            docs            LICENSE.txt             README.md                     reverted-merge       test.ps1
bad-commit                   bisect                    ff-merge        master-based-workflow   rebase-branch                 save-my-commit       test.sh
basic-branching              change-author             git-attributes  merge-conflict          rebase-exec                   SHELL-BASICS.md      testzsh.sh
basic-cherry-pick            commit-on-wrong-branch    git-tag         merge-driver            rebase-interactive-autosquash signed-commits       utils
basic-cleaning               commit-on-wrong-branch-2  ignore          merge-mergesort         rebase-multiple-commits       squashing
[ddnguyen@localhost git-katas]$ cd basic-staging
[ddnguyen@localhost basic-staging]$ ls
README.md  setup.ps1  setup.sh
```

```
# Git Kata: Basic Staging

This kata will examine the staging area of git.

In git we are working with three different areas:

* The working directory where you are making your changes
* The staging area where all changes you have added through `git add` will stay
* The repository where every commit ends up, making your history. To put your staged changes in here you issue the `git commit` command.

A file can have changes both in the working directory and staging area at the same time.
These changes do not have to be the same.

We will also work with `git restore` to restore the staged changes of a file, and `git checkout` to return a file to a previous state.

## Setup

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

You live in your own repository. There is a file called `file.txt`.

1. What's the content of `file.txt`?
2. Overwrite the content in `file.txt`: `echo 2 > file.txt` to change the state of your file in the working directory (or `sc file.txt '2'` in PowerShell)
3. What does `git diff` tell you?
4. What does `git diff --staged` tell you? why is this blank?
5. Run `git add file.txt` to stage your changes from the working directory.
6. What does `git diff` tell you?
7. What does `git diff --staged` tell you?
8. Overwrite the content in `file.txt`: `echo 3 > file.txt` to change the state of your file in the working directory (or `sc file.txt '3'` in PowerShell).
9. What does `git diff` tell you?
10. What does `git diff --staged` tell you?
11. Explain what is happening
12. Run `git status` and observe that `file.txt` are present twice in the output.
13. Run `git restore --staged file.txt` to unstage the change
14. What does `git status` tell you now?
15. Stage the change and make a commit
16. What does the log look like?
17. Overwrite the content in `file.txt`: `echo 4 > file.txt` (or `sc file.txt '4'` in PowerShell)
18. What is the content of `file.txt`?
19. What does `git status` tell us?
20. Run `git restore file.txt`
21. What is the content of `file.txt`?
22. What does `git status` tell us?

## Useful commands
```

Setup:

```
[ddnguyen@localhost basic-staging]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/git-katas/basic-staging/exercise/.git/
Successfully configured local username and email
Successfully disable local GPG signing
[master (root-commit) e6e19e3] 1
 1 file changed, 1 insertion(+)
 create mode 100644 file.txt
Successfully unset local username and email
Successfully unset local GPG signing
[ddnguyen@localhost exercise]$
```

Task 1:

```
[ddnguyen@localhost basic-staging]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/git-katas/basic-staging/exercise/.git/
Successfully configured local username and email
Successfully disable local GPG signing
[master (root-commit) e6e19e3] 1
 1 file changed, 1 insertion(+)
 create mode 100644 file.txt
Successfully unset local username and email
Successfully unset local GPG signing
[ddnguyen@localhost exercise]$ ls
file.txt
[ddnguyen@localhost exercise]$ cat file.txt
1
[ddnguyen@localhost exercise]$
```

Task 2:

```
[ddnguyen@localhost exercise]$ echo 2 > file.txt
[ddnguyen@localhost exercise]$ cat file.txt
2
[ddnguyen@localhost exercise]$
```

Task 3:

```
[ddnguyen@localhost exercise]$ git diff
diff --git a/file.txt b/file.txt
index d00491f..0cfbf08 100644
--- a/file.txt
+++ b/file.txt
@@ -1 +1 @@
-1
+2
[ddnguyen@localhost exercise]$
```

Task 4:

```
[ddnguyen@localhost exercise]$ git diff --staged
[ddnguyen@localhost exercise]$
```

- **Lệnh git diff --staged có chức năng so sánh giữa Staging Area và Repository (so sánh khu vực chờ với commit cuối cùng) vì chưa gõ lệnh git add để đưa sự thay đổi vào Staging Area, nên Staging Area lúc này hoàn toàn trống.**

Task 5:

```
[ddnguyen@localhost exercise]$ git add file.txt
[ddnguyen@localhost exercise]$
```

Task 6:

```
[ddnguyen@localhost exercise]$ git diff
[ddnguyen@localhost exercise]$
```

Task 7:

```
[ddnguyen@localhost exercise]$ git diff --staged
diff --git a/file.txt b/file.txt
index d00491f..0cfbf08 100644
--- a/file.txt
+++ b/file.txt
@@ -1 +1 @@
-1
+2
[ddnguyen@localhost exercise]$
```

Task 8:

```
[ddnguyen@localhost exercise]$ echo 3 > file.txt
[ddnguyen@localhost exercise]$ cat file.txt
3
[ddnguyen@localhost exercise]$
```

Task 9:

```
[ddnguyen@localhost exercise]$ git diff
diff --git a/file.txt b/file.txt
index 0cfbf08..00750ed 100644
--- a/file.txt
+++ b/file.txt
@@ -1 +1 @@
-2
+3
```

Task 10:

```
[ddnguyen@localhost exercise]$ git diff --staged
diff --git a/file.txt b/file.txt
index d00491f..0cfbf08 100644
--- a/file.txt
+++ b/file.txt
@@ -1 +1 @@
-1
+2
[ddnguyen@localhost exercise]$
```

Task 11:

Ở task 8 lệnh echo 3 > file.txt sẽ ghi đè nội dụng file.txt từ 2 thành 3, lệnh git diff ở task 9 sẽ so sánh sự biệt của tệp tin file.txt (-2), (+3), ở task 10 lệnh git diff –staged dùng để

so sánh sự khác biệt giữa các thay đổi đã được đưa vào vùng chờ (staging area) và lần commit gần nhất sau khi khi chạy lệnh git add (task 5).

Task 12:

```
[ddnguyen@localhost exercise]$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   file.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

[ddnguyen@localhost exercise]$
```

Task 13:

```
[ddnguyen@localhost exercise]$ git restore --staged file.txt
[ddnguyen@localhost exercise]$
```

Task 14:

```
[ddnguyen@localhost exercise]$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

no changes added to commit (use "git add" and/or "git commit -a")
[ddnguyen@localhost exercise]$
```

Task 15:

```
[ddnguyen@localhost exercise]$ git add file.txt
[ddnguyen@localhost exercise]$ git commit -m "Task 15"
[master 8d15ef1] Task 15
 1 file changed, 1 insertion(+), 1 deletion(-)
[ddnguyen@localhost exercise]$
```

Task 16:

```
[ddnguyen@localhost exercise]$ git log
commit 8d15ef14f8e7a09be3d07cd957fffdebbc5cc077 (HEAD -> master)
Author: dang <22124227@dlu.edu.vn>
Date:   Mon Mar 9 15:04:57 2026 +0700

    Task 15

commit e6e19e38ab4c973e6613eaa18577911e05dbfeca
Author: git-katas trainer bot <git-katas@example.com>
Date:   Mon Mar 9 14:39:02 2026 +0700

    1
[ddnguyen@localhost exercise]$
```

Task 17:

```
[ddnguyen@localhost exercise]$ echo 4 > file.txt
```

Task 18:

```
[ddnguyen@localhost exercise]$ echo 4 > file.txt
[ddnguyen@localhost exercise]$ cat file.txt
4
[ddnguyen@localhost exercise]$
```

Task 19:

```
[ddnguyen@localhost exercise]$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

no changes added to commit (use "git add" and/or "git commit -a")
[ddnguyen@localhost exercise]$
```

Task 20:

```
[ddnguyen@localhost exercise]$ git restore file.txt
[ddnguyen@localhost exercise]$
```

Task 21:

```
[ddnguyen@localhost exercise]$ cat file.txt
3
[ddnguyen@localhost exercise]$
```

Task 22:

```
[ddnguyen@localhost exercise]$ git status
On branch master
nothing to commit, working tree clean
[ddnguyen@localhost exercise]$
```

2.3 Investigation

```
[ddnguyen@localhost git-katas]$ ls
3-way-merge                  basic-commits             configure-git   images                  objects                         reorder-the-history  submodules
advanced-rebase-interactive  basic-revert              detached-head   investigation           Overview.md                     reset                subtree
alias                        basic-staging             diff-advance    lfs                     pre-push                        restore              test-all.sh
amend                        basic-stashing            docs            LICENSE.txt             README.md                       reverted-merge       test.ps1
bad-commit                   bisect                    ff-merge        master-based-workflow   rebase-branch                   save-my-commit       test.sh
basic-branching              change-author             git-attributes  merge-conflict          rebase-exec                     SHELL-BASICS.md      testzsh.sh
basic-cherry-pick            commit-on-wrong-branch    git-tag         merge-driver            rebase-interactive-autosquash   signed-commits       utils
basic-cleaning               commit-on-wrong-branch-2  ignore          merge-mergesort         rebase-multiple-commits         squashing
[ddnguyen@localhost git-katas]$ cd investigation
[ddnguyen@localhost investigation]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost investigation]$ cat README.md
# Git Kata: Git objects

Objects are stored in `<repository>/.git/objects` in subfolders matching the first two chars of the sha.
`fc1da6e8f` is therefore the file: `.git/objects/fc/1da6e8f`.

`git cat-file` inflates and shows the content of whatever _ref_ you pass it.
`-p` asks `cat-file` to pretty-print the content of an object.

`git ls-tree master .` inflates and lists the content of a folder.

## Setup:

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## Task

1. Using `git ls-tree` and `git cat-file`, draw the entire Git data structure.
        - What tree and blob objects do you have and what do they point at?
        - What commits point inside this graph and where?
[ddnguyen@localhost investigation]$
```

Setup:

```
[ddnguyen@localhost investigation]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/git-katas/investigation/exercise/.git/
[master (root-commit) ddbe797] add test.md
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test.md
[master 870b013] Add folders
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 src/file.c
 create mode 100644 src/main/main.h
[master 07f57ac] Put data in main.h
 1 file changed, 1 insertion(+)
[master ee0cd62] Fill test.md
 1 file changed, 1 insertion(+)
[ddnguyen@localhost exercise]$
```

Task 1:

```
[ddnguyen@localhost exercise]$ git ls-tree master
040000 tree 68ebdecef990e7a93b49666c59e237f7b961403a    src
100644 blob 3e01ae02c5a70d3a92fada56dc20457a6e4deb52    test.md
[ddnguyen@localhost exercise]$ ls
src  test.md
[ddnguyen@localhost exercise]$ cat src
cat: src: Is a directory
[ddnguyen@localhost exercise]$ git cat-file -p  68ebdecef990e7a93b49666c59e237f7b961403a
100644 blob e69de29bb2d1d6434b8b29ae775ad8c2e48c5391    file.c
040000 tree 7f36089284f5f040cc9f63b7998c931330d6e67a    main
[ddnguyen@localhost exercise]$ git cat-file -p 3e01ae02c5a70d3a92fada56dc20457a6e4deb52
This is also a bunch of data
[ddnguyen@localhost exercise]$ git cat-file -p  e69de29bb2d1d6434b8b29ae775ad8c2e48c5391
[ddnguyen@localhost exercise]$ git cat-file -p  e69de29bb2d1d6434b8b29ae775ad8c2e48c5391
[ddnguyen@localhost exercise]$ git cat-file -p  7f36089284f5f040cc9f63b7998c931330d6e67a
100644 blob 49f2298b586912944c24349543c47f077f633f25    main.h
[ddnguyen@localhost exercise]$ git cat-file -p  49f2298b586912944c24349543c47f077f633f25
This is more data
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

3:33 PM
3/9/2026

Master:

+ src:

+ file.c

+ main:

+main.h

+ test.md

2.4 Basic Branching

```
[ddnguyen@localhost git-katas]$ ls
3-way-merge                 basic-commits             configure-git   images                  objects                       reorder-the-history  submodules
advanced-rebase-interactive basic-revert              detached-head   investigation           Overview.md                   reset                subtree
alias                       basic-staging             diff-advance    lfs                     pre-push                      restore              test-all.sh
amend                       basic-stashing            docs            LICENSE.txt             README.md                     reverted-merge       test.ps1
bad-commit                  bisect                    ff-merge        master-based-workflow   rebase-branch                 save-my-commit       test.sh
basic-branching             change-author             git-attributes  merge-conflict          rebase-exec                   SHELL-BASICS.md      testzsh.sh
basic-cherry-pick           commit-on-wrong-branch    git-tag         merge-driver            rebase-interactive-autosquash signed-commits       utils
basic-cleaning              commit-on-wrong-branch-2  ignore          merge-mergesort         rebase-multiple-commits       squashing
[ddnguyen@localhost git-katas]$ cd basic-branching
[ddnguyen@localhost basic-branching]$ ls
README.md  setup.ps1  setup.sh
```

```
[ddnguyen@localhost basic-branching]$ cat README.md
# Git Kata: Basic Branching

## Setup

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

You again live in your own branch, this time we will be doing a bit of juggling with branches, to show how lightweight branches are in git.
Hint: `git switch` will make you switch from one branch to another.

1. Use `git branch` to see the two branches that are relevant for this exercise
2. What branch are you on?
3. Use `git branch mybranch` to create a new branch called _mybranch_
4. Use `git branch` again to see the new branch created.
5. Use `git switch mybranch` to switch to your new branch.
6. How does the output from `git status` change when you switch between the _master_ and the new branch that you have created?
7. How does the workspace change when you change between the two branches?
8. Make sure you are on your _mybranch_ branch before you continue.
9. Create a file called `file1.txt` with your name.
10. `Add` the file and `commit` with this change.
11. Use `git log --oneline --graph` to see your branch pointing to the new commit.
12. Switch back to the branch called _master_.
13. Use `git log --oneline --graph` and notice how the commit you made on the _mybranch_ branch is missing on the _master_ branch.
14. Make a new file called `file2.txt` and commit that file.
15. Use `git log --oneline --graph --all` to see your branch pointing to the new commit, and that the two branches now have different commits on them.
16. Switch to your branch _mybranch_.
17. What happened to your working directory? Can you see your `file2.txt`?
18. Use `git diff mybranch master` to see the difference between the two branches.

## Useful commands

- `git switch`
- `git switch -c`
- `git log --oneline --graph`
- `git branch`
- `git diff`
```

Setup:

```
[ddnguyen@localhost basic-branching]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/git-katas/basic-branching/exercise/.git/
Successfully configured local username and email
Successfully disable local GPG signing
[master (root-commit) 9f995db] dummy commit
 1 file changed, 1 insertion(+)
 create mode 100644 dummy.txt
Switched to a new branch 'second-branch'
Switched to branch 'master'
Successfully unset local username and email
Successfully unset local GPG signing
[ddnguyen@localhost exercise]$
```

Task 1:

```
[ddnguyen@localhost exercise]$ git branch
* master
  second-branch
[ddnguyen@localhost exercise]$
```

Task 2:

```
[ddnguyen@localhost exercise]$ git branch
* master
  second-branch
[ddnguyen@localhost exercise]$
```

Task 3:

```
[ddnguyen@localhost exercise]$ git branch mybranch
[ddnguyen@localhost exercise]$
```

Task 4:

```
[ddnguyen@localhost exercise]$ git branch
* master
  mybranch
  second-branch
[ddnguyen@localhost exercise]$
```

Task 5:

```
[ddnguyen@localhost exercise]$ git switch mybranch
Switched to branch 'mybranch'
[ddnguyen@localhost exercise]$
```

Task 6:

```
[ddnguyen@localhost exercise]$ git status
On branch mybranch
nothing to commit, working tree clean
[ddnguyen@localhost exercise]$
```

Task 7:

Task 8:

```
[ddnguyen@localhost exercise]$ git branch
  master
* mybranch
  second-branch
[ddnguyen@localhost exercise]$
```

Task 9:

```
  second-branch
[ddnguyen@localhost exercise]$ touch file1.txt
[ddnguyen@localhost exercise]$ ls
dummy.txt  file1.txt
[ddnguyen@localhost exercise]$
```

Task 10:

```
[ddnguyen@localhost exercise]$ git add file1.txt
[ddnguyen@localhost exercise]$ git commit -m "Task 10"
[mybranch 211ba3a] Task 10
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1.txt
[ddnguyen@localhost exercise]$
```

Task 11:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph
* 211ba3a (HEAD -> mybranch) Task 10
* 9f995db (second-branch, master) dummy commit
[ddnguyen@localhost exercise]$
```

Task 12:

```
[ddnguyen@localhost exercise]$ git switch master
Switched to branch 'master'
[ddnguyen@localhost exercise]$ git branch
* master
  mybranch
  second-branch
[ddnguyen@localhost exercise]$
```

Task 13:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph
* 9f995db (HEAD -> master, second-branch) dummy commit
[ddnguyen@localhost exercise]$
```

Task 14:

```
[ddnguyen@localhost exercise]$ touch file2.txt
[ddnguyen@localhost exercise]$ ls
dummy.txt  file2.txt
[ddnguyen@localhost exercise]$
```

Task 15:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph --all
* 211ba3a (mybranch) Task 10
* 9f995db (HEAD -> master, second-branch) dummy commit
[ddnguyen@localhost exercise]$
```

Task 16:

```
[ddnguyen@localhost exercise]$ git switch mybranch
Switched to branch 'mybranch'
[ddnguyen@localhost exercise]$ git branch
  master
* mybranch
  second-branch
[ddnguyen@localhost exercise]$
```

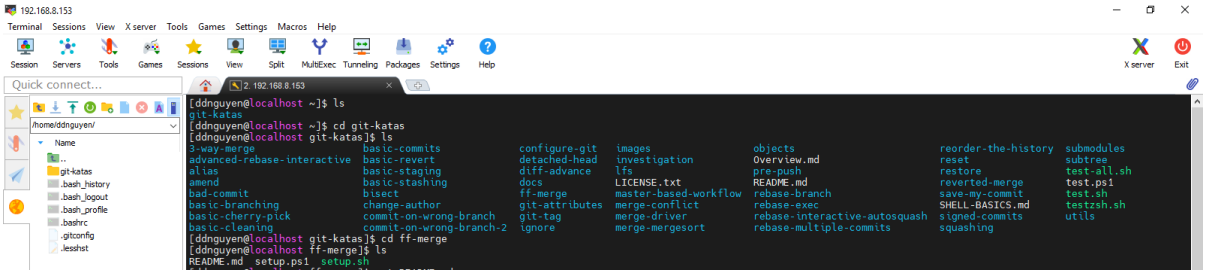
Task 17:

```
[ddnguyen@localhost exercise]$ ls
dummy.txt  file1.txt  file2.txt
[ddnguyen@localhost exercise]$
```

Task 18:

```
[ddnguyen@localhost exercise]$ git diff mybranch master
diff --git a/file1.txt b/file1.txt
deleted file mode 100644
index e69de29..0000000
[ddnguyen@localhost exercise]$
```

2.5 Fast-forward merge



```
[ddnguyen@localhost ff-merge]$ cat README.md
# Git Kata: Fast-forward Merge

## Setup

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

You again live in your own branch, this time we will be doing a bit of juggling with branches, to show how lightweight branches are in git.

1. Create a (feature)branch called `feature/uppercase` (yes, `feature/uppercase` is a perfectly legal branch name, and a common convention).
2. Switch to this branch
3. What is the output of `git status`?
4. Edit the greeting.txt to contain an uppercase greeting
5. Add `greeting.txt` files to staging area and commit
6. What is the output of `git branch`?
7. What is the output of `git log --oneline --graph --all`

   *Remember: You want to update the master branch so it also has all the changes currently on the feature branch. The command 'git merge [branch name]' takes one bra
nch as argument from which it takes changes. The branch pointed to by HEAD (currently checked out branch) is then updated to also include these changes.*

8. Switch to the `master` branch
9. Use `cat` to see the contents of the greetings
10. Diff the branches
11. Merge the branches
12. Use `cat` to see the contents of the greetings
13. Delete the uppercase branch

## Useful commands

- `git branch`
- `git branch <branch-name>`
- `git branch -d <branch-name>`
- `git switch`
- `git branch -v`
- `git add`
- `git commit`
- `git commit -m`
- `git merge <branch>`
- `git diff <branchA> <branchB>`
- `git log --oneline --graph --all`
[ddnguyen@localhost ff-merge]$
```

Setup:

```
[ddnguyen@localhost ff-merge]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/git-katas/ff-merge/exercise/.git/
[master (root-commit) 98429a7] Add file greeting.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 greeting.txt
[master f6fbf0e] Add content to greeting.txt
 1 file changed, 1 insertion(+)
[ddnguyen@localhost exercise]$
```

Task 1:

```
[ddnguyen@localhost exercise]$ git branch feature/uppercase
[ddnguyen@localhost exercise]$
```

Task 2:

```
[ddnguyen@localhost exercise]$ git switch feature/uppercase
Switched to branch 'feature/uppercase'
[ddnguyen@localhost exercise]$ git branch
* feature/uppercase
  master
[ddnguyen@localhost exercise]$
```

Task 3:

```
[ddnguyen@localhost exercise]$ git status
On branch feature/uppercase
nothing to commit, working tree clean
[ddnguyen@localhost exercise]$
```

Task 4:

```
[ddnguyen@localhost exercise]$ ls
greeting.txt
[ddnguyen@localhost exercise]$ cat greeting.txt
hello
[ddnguyen@localhost exercise]$ nano greeting.txt
[ddnguyen@localhost exercise]$ cat greeting.txt
HELLO
[ddnguyen@localhost exercise]$
```

Task 5:

```
[ddnguyen@localhost exercise]$ git add greeting.txt
[ddnguyen@localhost exercise]$ git commit -m "Task 5"
[feature/uppercase 85413e0] Task 5
 1 file changed, 1 insertion(+), 1 deletion(-)
[ddnguyen@localhost exercise]$
```

Task 6:

```
[ddnguyen@localhost exercise]$ git branch
* feature/uppercase
  master
```

Task 7:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph --all
* 85413e0 (HEAD -> feature/uppercase) Task 5
* f6fbf0e (master) Add content to greeting.txt
* 98429a7 Add file greeting.txt
```

Task 8:

```
[ddnguyen@localhost exercise]$ git switch master
Switched to branch 'master'
[ddnguyen@localhost exercise]$
```

Task 9:

```
[ddnguyen@localhost exercise]$ ls
greeting.txt
[ddnguyen@localhost exercise]$ cat greeting.txt
hello
[ddnguyen@localhost exercise]$
```

Task 10:

```
[ddnguyen@localhost exercise]$ git diff master feature/uppercase
diff --git a/greeting.txt b/greeting.txt
index ce01362..e427984 100644
--- a/greeting.txt
+++ b/greeting.txt
@@ -1 +1 @@
-hello
+HELLO
[ddnguyen@localhost exercise]$
```

Task 11:

```
[ddnguyen@localhost exercise]$ git merge feature/uppercase
Updating f6fbf0e..85413e0
Fast-forward
 greeting.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Task 12:

```
[ddnguyen@localhost exercise]$ cat greeting.txt
HELLO
[ddnguyen@localhost exercise]$
```

Task 13:

```
[ddnguyen@localhost exercise]$ git branch -d feature/uppercase
Deleted branch feature/uppercase (was 85413e0).
[ddnguyen@localhost exercise]$
```

2.6 3 way merge

```
[ddnguyen@localhost git-katas]$ ls
3-way-merge                  basic-commits            configure-git   images                  objects                       reorder-the-history  submodules
advanced-rebase-interactive  basic-revert             detached-head   investigation           Overview.md                   reset                subtree
alias                        basic-staging            diff-advance    lfs                     pre-push                      restore              test-all.sh
amend                        basic-stashing           docs            LICENSE.txt             README.md                     reverted-merge       test.ps1
bad-commit                   bisect                   ff-merge        master-based-workflow   rebase-branch                 save-my-commit       test.sh
basic-branching              change-author            git-attributes  merge-conflict          rebase-exec                   SHELL-BASICS.md      testzsh.sh
basic-cherry-pick            commit-on-wrong-branch   git-tag         merge-driver            rebase-interactive-autosquash signed-commits       utils
basic-cleaning               commit-on-wrong-branch-2 ignore          merge-mergesort         rebase-multiple-commits       squashing
[ddnguyen@localhost git-katas]$ cd 3-way-merge
[ddnguyen@localhost 3-way-merge]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost 3-way-merge]$ cat README.md
# Git Kata: 3-Way Merge

## Setup

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

You again live in your own branch, this time we will be doing a bit of juggling with branches, to show how lightweight branches are in git.

1. Create a branch called greeting and switch to it
2. Edit the greeting.txt to contain your favorite greeting
3. Add greeting.txt files to the staging area
4. Commit
5. Switch back to the master branch
6. Create a file README.md with information about this repository
7. Add the README.md file to staging area and make the commit
8. What is the output of `git log --oneline --graph --all`?
9. Diff the branches
10. Merge the greeting branch into master
11. What is the output of `git log --oneline --graph --all` now? Observe the extra merge commit created with the message "Merge branch 'greeting'".

## Useful commands

- `git branch`
- `git branch <branch-name>`
- `git branch -d <branch-name>`
- `git switch <branch-name>`
- `git switch -c <branch-name>`
- `git branch -v`
- `git add`
- `git commit`
- `git commit -m`
- `git merge <branch-name>`
```

```
- `git diff <branchA> <branchB>`
- `git log --oneline --graph --all`
```

Setup:

```
[ddnguyen@localhost 3-way-merge]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/git-katas/3-way-merge/exercise/.git/
[master (root-commit) e16fa03] Add file greeting.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 greeting.txt
[master de32f75] Add content to greeting.txt
 1 file changed, 1 insertion(+)
[ddnguyen@localhost exercise]$
```

Task 1:

```
[ddnguyen@localhost exercise]$ git branch greeting
[ddnguyen@localhost exercise]$ git switch greeting
Switched to branch 'greeting'
[ddnguyen@localhost exercise]$
```

Task 2:

```
[ddnguyen@localhost exercise]$ cat greeting.txt
hello
[ddnguyen@localhost exercise]$ nano greeting.txt
[ddnguyen@localhost exercise]$ cat greeting.txt
2212427_DangDangNguyen
[ddnguyen@localhost exercise]$
```

Task 3:

```
[ddnguyen@localhost exercise]$ git add greeting.txt
[ddnguyen@localhost exercise]$
```

Task 4:

```
[ddnguyen@localhost exercise]$ git commit -m "Task 4"
[greeting d48808d] Task 4
 1 file changed, 1 insertion(+), 1 deletion(-)
[ddnguyen@localhost exercise]$
```

Task 5:

```
[ddnguyen@localhost exercise]$ git switch master
Switched to branch 'master'
[ddnguyen@localhost exercise]$
```

Task 6:

```
[ddnguyen@localhost exercise]$ git switch master
Switched to branch 'master'
[ddnguyen@localhost exercise]$ touch README.md
[ddnguyen@localhost exercise]$ ls
greeting.txt  README.md
[ddnguyen@localhost exercise]$ nano  README.md
[ddnguyen@localhost exercise]$
```

Task 7:

```
[ddnguyen@localhost exercise]$ git add README.md
[ddnguyen@localhost exercise]$ git commit -m "Task 7"
[master 2c5f711] Task 7
 1 file changed, 27 insertions(+)
 create mode 100644 README.md
[ddnguyen@localhost exercise]$
```

Task 8:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph --all
* 2c5f711 (HEAD -> master) Task 7
| * d48808d (greeting) Task 4
|/
* de32f75 Add content to greeting.txt
* e16fa03 Add file greeting.txt
```

Task 9:

```
[ddnguyen@localhost exercise]$ git diff master greeting
diff --git a/README.md b/README.md
deleted file mode 100644
index e253c03..0000000
--- a/README.md
+++ /dev/null
@@ -1,27 +0,0 @@
-The task
-You again live in your own branch, this time we will be doing a bit of juggling with branches, to show how lightweight branches are in git.
-
-Create a branch called greeting and switch to it
-Edit the greeting.txt to contain your favorite greeting
-Add greeting.txt files to the staging area
-Commit
-Switch back to the master branch
-Create a file README.md with information about this repository
-Add the README.md file to staging area and make the commit
-What is the output of git log --oneline --graph --all?
-Diff the branches
-Merge the greeting branch into master
-What is the output of git log --oneline --graph --all now? Observe the extra merge commit created with the message "Merge branch 'greeting'".
-Useful commands
-git branch
-git branch <branch-name>
-git branch -d <branch-name>
-git switch <branch-name>
-git switch -c <branch-name>
-git branch -v
-git add
-git commit
-git commit -m
-git merge <branch-name>
-git diff <branchA> <branchB>
-git log --oneline --graph --all
diff --git a/greeting.txt b/greeting.txt
index ce01362..9485d5f 100644
--- a/greeting.txt
+++ b/greeting.txt
@@ -1 +1 @@
-hello
+2212427_DangDangNguyen
```

Task 10:

```
[ddnguyen@localhost exercise]$ git merge greeting
Merge made by the 'ort' strategy.
 greeting.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Taks 11:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph --all
*   bb84bd3 (HEAD -> master) Merge branch 'greeting'
|\
| * d48808d (greeting) Task 4
* | 2c5f711 Task 7
|/
* de32f75 Add content to greeting.txt
* e16fa03 Add file greeting.txt
```

2.7 Merge Mergesort

```
[ddnguyen@localhost git-katas]$ ls
3-way-merge                  basic-commits             configure-git   images                  objects                         reorder-the-history  submodules
advanced-rebase-interactive  basic-revert              detached-head   investigation           Overview.md                     reset                subtree
alias                        basic-staging             diff-advance    lfs                     pre-push                        restore              test-all.sh
amend                        basic-stashing            docs            LICENSE.txt             README.md                       reverted-merge       test.ps1
bad-commit                   bisect                    ff-merge        master-based-workflow   rebase-branch                   save-my-commit       test.sh
basic-branching              change-author             git-attributes  merge-conflict          rebase-exec                     SHELL-BASICS.md      testzsh.sh
basic-cherry-pick            commit-on-wrong-branch    git-tag         merge-driver            rebase-interactive-autosquash   signed-commits       utils
basic-cleaning               commit-on-wrong-branch-2  ignore          merge-mergesort         rebase-multiple-commits         squashing
[ddnguyen@localhost git-katas]$ cd merge-mergesort
[ddnguyen@localhost merge-mergesort]$ ls
base.py  lefty.py  README.md  righty.py  setup.ps1  setup.sh
[ddnguyen@localhost merge-mergesort]$ cat  README.md
# Git Kata: Merge Mergesort
In this kata you will be facing your first merge conflict!
There will be two different branches:

* Mergesort-Impl
* master

The task is to look at the merge conflict, and solve it by editing the file accordingly.

## Setup:

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

1. Run `git branch` to see the two branches present
2. Merge `Mergesort-Impl` into `master`
3. Either:
   1. Solve the merge conflict with your favorite editor and finish the merge (`git status` will tell you what to do), **or**
   2. Use `git mergetool --tool=emerge` (for emacs fans) or `git mergetool --tool=vimdiff` (for vim fans) and finish the merge (`git status` will tell you what to do)

## Relevant commands
- `git branch`
- `git merge`
- `git status`
- `git mergetool --tool=emerge`
- `git mergetool --tool=vimdiff`
- `git add`
- `git commit`

[ddnguyen@localhost merge-mergesort]$
```

baXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Setup:

```
[ddnguyen@localhost merge-mergesort]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/git-katas/merge-mergesort/exercise/.git/
[master (root-commit) 8126490] Fake it till you make it - Faking mergesort using python .sort() method
 1 file changed, 5 insertions(+)
 create mode 100644 mergesort.py
Switched to a new branch 'Mergesort-Impl'
[Mergesort-Impl 16a9304] Mergesort implemented on feature branch
 1 file changed, 43 insertions(+), 3 deletions(-)
Switched to branch 'master'
[master ba4071c] Mergesort implemented on master
 1 file changed, 43 insertions(+), 3 deletions(-)
[ddnguyen@localhost exercise]$
```

Task 1:

```
[ddnguyen@localhost exercise]$ git branch
  Mergesort-Impl
* master
[ddnguyen@localhost exercise]$
```

Task 2:

```
[ddnguyen@localhost exercise]$ git merge Mergesort-Impl
Auto-merging mergesort.py
CONFLICT (content): Merge conflict in mergesort.py
Automatic merge failed; fix conflicts and then commit the result.
```

Task 3:

```
[ddnguyen@localhost exercise]$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   mergesort.py

no changes added to commit (use "git add" and/or "git commit -a")
[ddnguyen@localhost exercise]$ git add  mergesort.py
[ddnguyen@localhost exercise]$ git merge Mergesort-Impl
fatal: You have not concluded your merge (MERGE_HEAD exists).
Please, commit your changes before you merge.
[ddnguyen@localhost exercise]$ fatal: You have not concluded your merge (MERGE_HEAD exists).
Please, commit your changes before you merge.
fatal: You have not concluded your merge (MERGE_HEAD exists).
Please, commit your changes before you merge.
^C
[ddnguyen@localhost exercise]$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   mergesort.py

[ddnguyen@localhost exercise]$ git commut
git: 'commut' is not a git command. See 'git --help'.

The most similar command is
        commit
[ddnguyen@localhost exercise]$ git commit
[master 9b17eb1] Merge branch 'Mergesort-Impl'
[ddnguyen@localhost exercise]$ git commit
On branch master
nothing to commit, working tree clean
[ddnguyen@localhost exercise]$ git merge Mergesort-Impl
Already up to date.
[ddnguyen@localhost exercise]$ git merge Mergesort-Impl
Already up to date.
[ddnguyen@localhost exercise]$ ls
mergesort.py
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

2.8 Rebase Branch

```
[ddnguyen@localhost git-katas]$ ls
3-way-merge                basic-commits             configure-git   images                  objects                       reorder-the-history  submodules
advanced-rebase-interactive basic-revert             detached-head   investigation           Overview.md                   reset                subtree
alias                      basic-staging             diff-advance    lfs                     pre-push                      restore              test-all.sh
amend                      basic-stashing            docs            LICENSE.txt             README.md                     reverted-merge       test.ps1
bad-commit                 bisect                    ff-merge        master-based-workflow   rebase-branch                 save-my-commit       test.sh
basic-branching            change-author             git-attributes  merge-conflict          rebase-exec                   SHELL-BASICS.md      testzsh.sh
basic-cherry-pick          commit-on-wrong-branch    git-tag         merge-driver            rebase-interactive-autosquash signed-commits       utils
basic-cleaning             commit-on-wrong-branch-2  ignore          merge-mergesort         rebase-multiple-commits       squashing
[ddnguyen@localhost git-katas]$ cd rebase-branch
[ddnguyen@localhost rebase-branch]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost rebase-branch]$ cat README.md
# Git Kata: rebase branch

## Setup

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

You again live in your own branch, this time we will be doing a bit of juggling with branches, to show how lightweight branches are in git.

1. Which branches exist?
2. Look at the log for the master branch
3. Switch to the uppercase branch
4. How does the log compare to the log on the master branch?
5. Rebase your uppercase branch with the master (`git rebase master`)
6. What did just happen? Draw it!
7. Now switch to the master branch
8. Merge uppercase into master
9. What does the log look like now?

## Useful commands

- `git switch <branch-name>`
- `git rebase <branch-name>`
- `git log --oneline --graph --all`
- `git merge <branch-name>`
[ddnguyen@localhost rebase-branch]$
```

Setup:

```
[ddnguyen@localhost rebase-branch]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/git-katas/rebase-branch/exercise/.git/
[master (root-commit) 35115d4] Add file greeting.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 greeting.txt
[master 425bbd7] Add content to greeting.txt
 1 file changed, 1 insertion(+)
Switched to a new branch 'uppercase'
[uppercase 3a659d9] Change greeting to uppercase
 1 file changed, 1 insertion(+), 1 deletion(-)
Switched to branch 'master'
[master 103c297] Add readme
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
[ddnguyen@localhost exercise]$
```

Task 1:

```
[ddnguyen@localhost exercise]$ git branch
* master
  uppercase
[ddnguyen@localhost exercise]$
```

Task 2:

```
[ddnguyen@localhost exercise]$ git log
commit 103c2978cb075736de6693736c7e4471ff83381c (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Add readme

commit 425bbd7cc434e77ef7025f4ecdd6d9296de791ec
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Add content to greeting.txt

commit 35115d4974dbea769ccbc2546e01d255b8e71ad7
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Add file greeting.txt
[ddnguyen@localhost exercise]$
```

Task 3:

```
[ddnguyen@localhost exercise]$ git switch uppercase
Switched to branch 'uppercase'
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 4:

```
[ddnguyen@localhost exercise]$ git log
commit 3a659d921c65d9ef7cf1c5dfb6e6edccf1903c24 (HEAD -> uppercase)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Change greeting to uppercase

commit 425bbd7cc434e77ef7025f4ecdd6d9296de791ec
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Add content to greeting.txt

commit 35115d4974dbea769ccbc2546e01d255b8e71ad7
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Add file greeting.txt
```

Task 5:

```
[ddnguyen@localhost exercise]$ git rebase master
Successfully rebased and updated refs/heads/uppercase.
```

Task 6:

Task 7:

```
[ddnguyen@localhost exercise]$ git switch master
Switched to branch 'master'
[ddnguyen@localhost exercise]$
```

Task 8:

```
[ddnguyen@localhost exercise]$ git merge uppercase
Updating 103c297..2da2ebd
Fast-forward
 greeting.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
[ddnguyen@localhost exercise]$
```

Taks 9:

```
[ddnguyen@localhost exercise]$ git log
commit 2da2ebd55f24ec4c05cf2f1cfc63e0f9a66c805d (HEAD -> master, uppercase)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Change greeting to uppercase

commit 103c2978cb075736de6693736c7e4471ff83381c
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Add readme

commit 425bbd7cc434e77ef7025f4ecdd6d9296de791ec
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Add content to greeting.txt

commit 35115d4974dbea769ccbc2546e01d255b8e71ad7
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 08:48:30 2026 +0700

    Add file greeting.txt
```

2.9 Basic revert

```
[ddnguyen@localhost git-katas]$ cd basic-revert
[ddnguyen@localhost basic-revert]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost basic-revert]$ cat README.md
# Git Kata: Basic revert
## Setup:

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

In this task a few changes snuck in, that we'd like to get out. Our history is public, so we can't just change it. Rather we need to use revert to remove the unwanted
 changes in a safe way.

1. Use `git log --oneline` to look at the history
2.  Use `cat` to view the content of `greeting.txt`
3.  Use `git revert` on the newest commit, to remove the changes the last commit added
4.  Use `git log --oneline` to view the history
5.  Did the revert command add or remove a commit?
6.  Use `cat` to view the content of `greeting.txt`
7.  Use `ls` to see the content of the workspace
8.  Use `git log --oneline` to find the sha of the commit adding credentials to the repository
9.  Use `git revert` to revert the commit that added the credentials
10. Use `git log --oneline` to view the history
11. Use `ls` to see the content of the workspace
12. How many commits were added or changed by the last revert?
13. Use `git show` with the sha of the commit you reverted to see that the credentials file is stilll in the history
14. As you have now reverted the credentials file, so it is removed from your working directory, is it also removed from git?

## Useful commands
- `git revert <ref>`
- `git log --oneline`
- `git show <ref>`
[ddnguyen@localhost basic-revert]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Setup:

```
[ddnguyen@localhost basic-revert]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/basic-revert/exercise/.git/
Successfully configured local username and email
Successfully disable local GPG signing
[master (root-commit) 06f4d01] Add file greeting.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 greeting.txt
[master 70474af] Add credentials to repository
 1 file changed, 1 insertion(+)
 create mode 100644 credentials.txt
[master e4b4dfa] Add content to greeting.txt
 1 file changed, 1 insertion(+)
[master 5fc4c91] Overwrite greeting.txt
 1 file changed, 1 insertion(+), 1 deletion(-)
Successfully unset local username and email
Successfully unset local GPG signing
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 1:

```
[ddnguyen@localhost exercise]$ git log --oneline
5fc4c91 (HEAD -> master) Overwrite greeting.txt
e4b4dfa Add content to greeting.txt
70474af Add credentials to repository
06f4d01 Add file greeting.txt
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 2:

```
[ddnguyen@localhost exercise]$ cat greeting.txt
This should have been appended to the original content, rather than overwriting it.
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 3:

```
[ddnguyen@localhost exercise]$ git revert HEAD
[master 87dacb1] Revert "Overwrite greeting.txt"
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Task 4:

```
[ddnguyen@localhost exercise]$ git log --oneline
87dacb1 (HEAD -> master) Revert "Overwrite greeting.txt"
5fc4c91 Overwrite greeting.txt
e4b4dfa Add content to greeting.txt
70474af Add credentials to repository
06f4d01 Add file greeting.txt
```

Task 5:

Task 6:

```
[ddnguyen@localhost exercise]$ ls
credentials.txt  greeting.txt
[ddnguyen@localhost exercise]$ cat greeting.txt
Original file content
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 7:

```
[ddnguyen@localhost exercise]$ ls
credentials.txt  greeting.txt
[ddnguyen@localhost exercise]$
```

Task 8:

```
[ddnguyen@localhost exercise]$ git log --oneline
87dacb1 (HEAD -> master) Revert "Overwrite greeting.txt"
5fc4c91 Overwrite greeting.txt
e4b4dfa Add content to greeting.txt
70474af Add credentials to repository
06f4d01 Add file greeting.txt
[ddnguyen@localhost exercise]$
```

Task 9:

```
[ddnguyen@localhost exercise]$ git revert 70474af
[master 7866ec8] Revert "Add credentials to repository"
 1 file changed, 1 deletion(-)
 delete mode 100644 credentials.txt
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 10:

```
[ddnguyen@localhost exercise]$ git log --oneline
7866ec8 (HEAD -> master) Revert "Add credentials to repository"
87dacb1 Revert "Overwrite greeting.txt"
5fc4c91 Overwrite greeting.txt
e4b4dfa Add content to greeting.txt
70474af Add credentials to repository
06f4d01 Add file greeting.txt
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

10:35 AM
3/11/2026

Task 11:

```
[ddnguyen@localhost exercise]$ ls
greeting.txt
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 12:

1

Task 13:

```
[ddnguyen@localhost exercise]$ git show
commit 7866ec80dd33170e35fff2e372da70a9ea5fa26b (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:34:36 2026 +0700

    Revert "Add credentials to repository"

    This reverts commit 70474afc38caa18c8ceca8607f3903d7f9fe3d33.

diff --git a/credentials.txt b/credentials.txt
deleted file mode 100644
index 8995708..0000000
--- a/credentials.txt
+++ /dev/null
@@ -1 +0,0 @@
-supersecretpassword
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 14:

No

2.10 Reset

```
[ddnguyen@localhost git-katas]$ cd reset
[ddnguyen@localhost reset]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost reset]$ cat README.md
# Git katas: Git reset
We can manipulate the History very much so. We should only ever tinker with our local history. As publicly release commits must expect to be immutable.

We use reset to unstage change, but we can also do many more different things.

## Setup

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## Task

1. How does your working directory look like?
2. What does your log look like? What does your stage look like?
3. Try to run `git reset --soft HEAD~1`
4. What happens to your working directory, your log and your stage?
5. Run `git reset --mixed HEAD~1`
6. What happens to your working directory, your log and your stage?
7. Run `git reset --hard HEAD~1`
8. What happens to your working directory, your log and your stage?
9. Now try to use `git revert HEAD~1`
10. What happens to your working directory, your log and your stage?

## Useful commands

- `git log --oneline`
- `git commit --amend`
- `git status`
- `git reset --soft`
- `git reset --mixed`
- `git reset --hard`
- `git revert`

## Further explanation

The following is taken from Recap section of [https://git-scm.com/book/en/v2/Git-Tools-Reset-Demystified].
The reset command overwrites these three trees in a specific order, stopping when you tell it to:
1. Move what the branch HEAD points to (stop here if --soft)
2. Make the stage look like HEAD (stop here unless --hard)
3. Make the working directory look like the stage
[ddnguyen@localhost reset]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Setup:

```
## Further explanation

The following is taken from Recap section of [https://git-scm.com/book/en/v2/Git-Tools-Reset-Demystified].
The reset command overwrites these three trees in a specific order, stopping when you tell it to:
1. Move what the branch HEAD points to (stop here if --soft)
2. Make the stage look like HEAD (stop here unless --hard)
3. Make the working directory look like the stage
[ddnguyen@localhost reset]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/reset/exercise/.git/
[master (root-commit) e2eb7df] 1
 1 file changed, 1 insertion(+)
 create mode 100644 1.txt
[master 3d1767b] 2
 1 file changed, 1 insertion(+)
 create mode 100644 2.txt
[master 78f3d91] 3
 1 file changed, 1 insertion(+)
 create mode 100644 3.txt
[master 60ddc82] 4
 1 file changed, 1 insertion(+)
 create mode 100644 4.txt
[master 24bbd40] 5
 1 file changed, 1 insertion(+)
 create mode 100644 5.txt
[master 19fe24a] 6
 1 file changed, 1 insertion(+)
 create mode 100644 6.txt
[master 7784994] 7
 1 file changed, 1 insertion(+)
 create mode 100644 7.txt
[master 92b8a80] 8
 1 file changed, 1 insertion(+)
 create mode 100644 8.txt
[master 19328ba] 9
 1 file changed, 1 insertion(+)
 create mode 100644 9.txt
[master 587e1a5] 10
 1 file changed, 1 insertion(+)
 create mode 100644 10.txt
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 1:

```
[ddnguyen@localhost exercise]$ ls
10.txt  1.txt  2.txt  3.txt  4.txt  5.txt  6.txt  7.txt  8.txt  9.txt
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

10:39 AM

Task 2:

```
[ddnguyen@localhost exercise]$ git log
commit 587e1a5efb98d00dbbe4c16338076f5ce120e5d6 (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    10

commit 19328ba18871153a4fb695c02d391989fa4ebdd1
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    9

commit 92b8a8084b23c5f6f8b86cd776bcf1338aae56d9
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    8

commit 7784994108e991dd2012359e01677daf36a996e7
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    7

commit 19fe24af69133c15c4966f665b9583a45dcab7d6
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    6

commit 24bbd405548a9b81310299c8e4aa2ec380bb62a3
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    5

commit 60ddc82624b782ab82d47677ac0d0afb143e1c7e
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    4

commit 78f3d911dfb22ff99d8f87bb9454a527566866df
Author: dang <2212427@dlu.edu.vn>
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

10:39 AM
3/11/2026

Task 3:

```
[ddnguyen@localhost exercise]$ git reset --soft HEAD~1
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

10:40 AM
3/11/2026

Task 4:

```
[ddnguyen@localhost exercise]$ git reset --soft HEAD~1
[ddnguyen@localhost exercise]$ ls
 10.txt   1.txt   2.txt   3.txt   4.txt   5.txt   6.txt   7.txt   8.txt   9.txt  'et --soft HEAD~1'
[ddnguyen@localhost exercise]$ git log
commit 19328ba18871153a4fb695c02d391989fa4ebdd1 (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    9

commit 92b8a8084b23c5f6f8b86cd776bcf1338aae56d9
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    8

commit 7784994108e991dd2012359e01677daf36a996e7
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    7

commit 19fe24af69133c15c4966f665b9583a45dcab7d6
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    6

commit 24bbd405548a9b81310299c8e4aa2ec380bb62a3
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    5

commit 60ddc82624b782ab82d47677ac0d0afb143e1c7e
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    4
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 5:

```
[ddnguyen@localhost exercise]$ git reset --mixed HEAD~1
[ddnguyen@localhost exercise]$ ls
 10.txt   1.txt   2.txt   3.txt   4.txt   5.txt   6.txt   7.txt   8.txt   9.txt  'et --mixed HEAD~1'  'et --soft HEAD~1'
[ddnguyen@localhost exercise]$
```

baXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 6:

```
    4

commit 78f3d911dfb22ff99d8f87bb9454a527566866df
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    3

commit 3d1767bb019bd76a16ee761be40399d2e7e8efe0
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

    2

commit e2eb7dfe1b798bc015b40db2c01522b1237e85f8
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 10:38:37 2026 +0700

:
```

aXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 7:

```
[ddnguyen@localhost exercise]$ git reset --hard HEAD~1
HEAD is now at 7784994 7
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 8:

```
[ddnguyen@localhost exercise]$ ls
 10.txt   1.txt   2.txt   3.txt   4.txt   5.txt   6.txt   7.txt   9.txt  'et --mixed HEAD~1'  'et --soft HEAD~1'
[ddnguyen@localhost exercise]$
```

Task 9:

```
[ddnguyen@localhost exercise]$ git revert HEAD~1
[master 0f955ab] Revert "6"
```

baXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 10:

```
 delete mode 100644 6.txt
[ddnguyen@localhost exercise]$ ls
 10.txt   1.txt   2.txt   3.txt   4.txt   5.txt   7.txt   9.txt  'et --mixed HEAD~1'  'et --soft HEAD~1'
[ddnguyen@localhost exercise]$
```

MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

2.10 Basic cleaning

```
[ddnguyen@localhost git-katas]$ cd basic-cleaning
[ddnguyen@localhost basic-cleaning]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost basic-cleaning]$ cat README.md
# Git Kata: Basic cleaning

## Setup:

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task
You are working on a project that involves generated files.  Say you are compiling C files into object files. Before checking out a new branch you want to start clean

1. Explore the directory with `ls -R`. There is a lot going on.  Code files, temp files, object files,..  Let's clean up!
2. Just to be safe, do a dry run and execute the clean command with the ` -n` option
3. Oh noes!  there's a `.c` file that would have been deleted!
4. Add `src/mylib.c` to the staging area. don't commit it.
5. Run the clean command with the ` -n` option. Notice that mylib.c will not be deleted. Also notice that the files in the obj directory are not listed
6. Run the clean command with the ` -n -d ` option.
7. Looks good! clean the repo ` -f -d `

## Useful commands
- `git clean -n`
- `git add`
- `git clean -n -d`
- `git clean -f -d`
[ddnguyen@localhost basic-cleaning]$
```

MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Setup:

```
[ddnguyen@localhost basic-cleaning]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/basic-cleaning/exercise/.git/
[master (root-commit) cc2e6a3] C code for my app
 3 files changed, 3 insertions(+)
 create mode 100644 README.txt
 create mode 100644 src/myapp.c
 create mode 100644 src/myapp.h
[ddnguyen@localhost exercise]$
```

MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 1:

```
[ddnguyen@localhost exercise]$ ls -R
.:
obj  README.txt  README.txt~  src

./obj:
a.out  myapp.o  mylib.a  mylib.o

./src:
myapp.c  myapp.c~  myapp.h  mylib.c  oldfile.c~
[ddnguyen@localhost exercise]$
```

MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 2:

```
[ddnguyen@localhost exercise]$ git clean -n
Would remove README.txt~
Would remove src/myapp.c~
Would remove src/mylib.c
Would remove src/oldfile.c~
[ddnguyen@localhost exercise]$
```

Task 3:

Task 4:

```
[ddnguyen@localhost exercise]$ git add src/mylib.c
[ddnguyen@localhost exercise]$
```

MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 5:

```
[ddnguyen@localhost exercise]$ git clean -n
Would remove README.txt~
Would remove src/myapp.c~
Would remove src/oldfile.c~
[ddnguyen@localhost exercise]$
```

Task 6:

```
[ddnguyen@localhost exercise]$ git clean -n -d
Would remove README.txt~
Would remove obj/
Would remove src/myapp.c~
Would remove src/oldfile.c~
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 7:

```
[ddnguyen@localhost exercise]$ git clean -f -d
Removing README.txt~
Removing obj/
Removing src/myapp.c~
Removing src/oldfile.c~
[ddnguyen@localhost exercise]$
```

2.11 Amend

```
[ddnguyen@localhost git-katas]$ ls
3-way-merge                 basic-commits             configure-git   images                  objects                         reorder-the-history  submodules
advanced-rebase-interactive basic-revert              detached-head   investigation           Overview.md                     reset                subtree
alias                       basic-staging             diff-advance    lfs                     pre-push                        restore              test-all.sh
amend                       basic-stashing            docs            LICENSE.txt             README.md                       reverted-merge       test.ps1
bad-commit                  bisect                    ff-merge        master-based-workflow   rebase-branch                   save-my-commit       test.sh
basic-branching             change-author             git-attributes  merge-conflict          rebase-exec                     SHELL-BASICS.md      testzsh.sh
basic-cherry-pick           commit-on-wrong-branch    git-tag         merge-driver            rebase-interactive-autosquash   signed-commits       utils
basic-cleaning              commit-on-wrong-branch-2  ignore          merge-mergesort         rebase-multiple-commits         squashing
[ddnguyen@localhost git-katas]$ cd amend
[ddnguyen@localhost amend]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost amend]$ cat README.md
# Git kata: Amending commits
When we are working, we make a lot of commits.
Sometimes we just forget something obvious that we want to fix quickly.

`git commit --amend` allows us to do that - tinker with the last commit we made.

You can use `git log -p` or `git show` to inspect the contents of commits and file changes that were added to the commits.

## Setup:

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

1. What does `git status` tell us?
2. What does `git log -p` tell us?
3. Stage the addition of bar.txt
4. Run `git commit --amend`
5. What happened? What does `git log -p` tell us?
6. What happens if you run `git commit --amend` again?

## Useful commands

- `git add`
- `git log --oneline --graph`
- `git log -p`
- `git show`
- `git commit --amend`
[ddnguyen@localhost amend]$
```

Setup:

```
[ddnguyen@localhost amend]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/amend/exercise/.git/
[master (root-commit) 3629895] feature 73
 1 file changed, 1 insertion(+)
 create mode 100644 foo.txt
[ddnguyen@localhost exercise]$
```

Task 1:

```
[ddnguyen@localhost exercise]$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        bar.txt

nothing added to commit but untracked files present (use "git add" to track)
[ddnguyen@localhost exercise]$
```

Task 2:

```
[ddnguyen@localhost exercise]$ git log -p
commit 36298953f23d2a9a07a110b0d0700822d04e7ed0 (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:15:09 2026 +0700

    feature 73

diff --git a/foo.txt b/foo.txt
new file mode 100644
index 0000000..257cc56
--- /dev/null
+++ b/foo.txt
@@ -0,0 +1 @@
+foo
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 3:

```
[ddnguyen@localhost exercise]$ git add bar.txt
[ddnguyen@localhost exercise]$
```

Task 4:

```
[ddnguyen@localhost exercise]$ git commit --amend
[master 497c489] feature 73
 Date: Wed Mar 11 13:15:09 2026 +0700
 2 files changed, 2 insertions(+)
 create mode 100644 bar.txt
 create mode 100644 foo.txt
[ddnguyen@localhost exercise]$
```

oaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 5:

```
[ddnguyen@localhost exercise]$ git log -p
commit 497c489668c2b1bc92a5c5beac4a181655ee2db4 (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:15:09 2026 +0700

    feature 73

diff --git a/bar.txt b/bar.txt
new file mode 100644
index 0000000..5716ca5
--- /dev/null
+++ b/bar.txt
@@ -0,0 +1 @@
+bar
diff --git a/foo.txt b/foo.txt
new file mode 100644
index 0000000..257cc56
--- /dev/null
+++ b/foo.txt
@@ -0,0 +1 @@
+foo
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 6:

```
[ddnguyen@localhost exercise]$ git commit --amend
[master 6c063c9] feature 73
 Date: Wed Mar 11 13:15:09 2026 +0700
 2 files changed, 2 insertions(+)
 create mode 100644 bar.txt
 create mode 100644 foo.txt
[ddnguyen@localhost exercise]$
```

2.12 Recorder of history

```
[ddnguyen@localhost git-katas]$ cd reorder-the-history
[ddnguyen@localhost reorder-the-history]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost reorder-the-history]$ cat README.md
# gitkatas
## Kata 7: Reordering history
The commits here have obviously been made by a mad man.
Unfortunately they actually contain useful information - it's just that the history is weird.
You should fix this such that our `git log` looks great!

## Setup

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## Task

Reorder the history such that it actually makes sense - add the files in the order that matches their name.

1. Use `git log --oneline --graph` to view the commits
2. Also try `git reflog` to view the commits. `git reflog` defaults to `git reflog show` and this is an alias for `git log -g --abbrev-commit --pretty=oneline`
3. Use `git rebase -i <after-this-commit>` to reorder the commits. There are commments in the file you edit that explain the commands available.
4. Use `git log --oneline --graph` to view the result

### useful commands

- `git rebase -i <after-this-commit>`
- `git log --oneline --graph`
- `git reflog`
[ddnguyen@localhost reorder-the-history]$
```

Setup:

```
[ddnguyen@localhost reorder-the-history]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/reorder-the-history/remote/
Cloning into 'exercise'...
warning: You appear to have cloned an empty repository.
done.
[master (root-commit) 51c1687] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 foo.txt
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 214 bytes | 214.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/reorder-the-history/./remote
 * [new branch]      master -> master
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 155 bytes | 155.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/reorder-the-history/./remote
 * [new tag]         START -> START
[master 056653f] file1
 1 file changed, 1 insertion(+)
 create mode 100644 file1
[master 8c2f73b] file9
 1 file changed, 1 insertion(+)
 create mode 100644 file9
[master f5f39c8] file8
 1 file changed, 1 insertion(+)
 create mode 100644 file8
[master c83843c] file3
 1 file changed, 1 insertion(+)
 create mode 100644 file3
[master 8ac0b81] file4
 1 file changed, 1 insertion(+)
 create mode 100644 file4
[master ecbf7ef] file5
 1 file changed, 1 insertion(+)
 create mode 100644 file5
[master 6676937] file2
 1 file changed, 1 insertion(+)
 create mode 100644 file2
[master 7aa3b61] file6
 1 file changed, 1 insertion(+)
 create mode 100644 file6
[master 599370b] file7
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 1:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph
* 599370b (HEAD -> master) file7
* 7aa3b61 file6
* 6676937 file2
* ecbf7ef file5
* 8ac0b81 file4
* c83843c file3
* f5f39c8 file8
* 8c2f73b file9
* 056653f file1
* 51c1687 (tag: START, origin/master) Initial commit
[ddnguyen@localhost exercise]$
```

baXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 2:

```
[ddnguyen@localhost exercise]$ git reflog
599370b (HEAD -> master) HEAD@{0}: commit: file7
7aa3b61 HEAD@{1}: commit: file6
6676937 HEAD@{2}: commit: file2
ecbf7ef HEAD@{3}: commit: file5
8ac0b81 HEAD@{4}: commit: file4
c83843c HEAD@{5}: commit: file3
f5f39c8 HEAD@{6}: commit: file8
8c2f73b HEAD@{7}: commit: file9
056653f HEAD@{8}: commit: file1
51c1687 (tag: START, origin/master) HEAD@{9}: commit (initial): Initial commit
[ddnguyen@localhost exercise]$
```

Task 3:

```
[ddnguyen@localhost exercise]$ git reflog
599370b (HEAD -> master) HEAD@{0}: commit: file7
7aa3b61 HEAD@{1}: commit: file6
6676937 HEAD@{2}: commit: file2
ecbf7ef HEAD@{3}: commit: file5
8ac0b81 HEAD@{4}: commit: file4
c83843c HEAD@{5}: commit: file3
f5f39c8 HEAD@{6}: commit: file8
8c2f73b HEAD@{7}: commit: file9
056653f HEAD@{8}: commit: file1
51c1687 (tag: START, origin/master) HEAD@{9}: commit (initial): Initial commit
[ddnguyen@localhost exercise]$ git rebase -i <after-this-commit>
-bash: syntax error near unexpected token `newline'
[ddnguyen@localhost exercise]$ git rebase -i 7aa3b61
Successfully rebased and updated refs/heads/master.
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 4:

```
[ddnguyen@localhost exercise]$ git rebase -i 7aa3b61
Successfully rebased and updated refs/heads/master.
[ddnguyen@localhost exercise]$ git log --oneline --graph
* 599370b (HEAD -> master) file7
* 7aa3b61 file6
* 6676937 file2
* ecbf7ef file5
* 8ac0b81 file4
* c83843c file3
* f5f39c8 file8
* 8c2f73b file9
* 056653f file1
* 51c1687 (tag: START, origin/master) Initial commit
[ddnguyen@localhost exercise]$
```

2.13

```
[ddnguyen@localhost git-katas]$ cd advanced-rebase-interactive
[ddnguyen@localhost advanced-rebase-interactive]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost advanced-rebase-interactive]$ cat README.md
# gitkatas
## Advanced interactive rebase
You have worked on a new feature called Hello World.
This features ends up being complete with both documentation and unit test, but there are a few problems.
The history looks really messy, with lots of small half-finished steps, and there are things included that should never have been there.

You should fix this such that your `git log` looks great!

To do this we will use our good friend `git rebase --interactive`

Luckily we have a release tag `v0.0` from just before we started the feature.

As this is an advanced exercise, there are no specific steps to follow and no single solution.

## Setup:

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## Task

1. Explore the repo and the history so you know what happened
2. Use `git rebase --interactive v0.0` to let you edit the "recipe" for the entire feature development.
3. Clean up the history such that it actually makes sense. Try to use as many of the rebase "features" (e.g. reword, squash, fixup, drop) as possible. You decide your
self if you want to rewrite the whole thing in one go, or apply a few changes first, then run a new `git rebase --interactive v0.0` to keep cleaning.

### useful commands

- `ls -l`                # list files
- `tail -n +1 *`         # show content of all files
- `git log --oneline`    # show history
- `git log --stat`       # log which files changed
- `git log --patch`      # log with diff
- `git rebase -i <ref>`  # run the interactive rebase back to <ref>
[ddnguyen@localhost advanced-rebase-interactive]$
```

aXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

1:38 PM
3/11/2026

Setup:

```
[ddnguyen@localhost advanced-rebase-interactive]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/advanced-rebase-interactive/remote/
Cloning into 'exercise'...
warning: You appear to have cloned an empty repository.
done.
[master (root-commit) 4dd06a2] Initial commit
Enumerating objects: 2, done.
Counting objects: 100% (2/2), done.
Writing objects: 100% (2/2), 160 bytes | 160.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/advanced-rebase-interactive/./remote
 * [new branch]      master -> master
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/advanced-rebase-interactive/./remote
 * [new tag]         v0.0 -> v0.0
[master 6fad267] Helo Volrd feature
 1 file changed, 1 insertion(+)
 create mode 100644 hello.code
[master e158e59] Finished HW feature
 2 files changed, 2 insertions(+), 1 deletion(-)
 create mode 100644 other.code
[master 3b4fd61] Really made the thingy done
 1 file changed, 1 insertion(+), 1 deletion(-)
[master 92147ce] debugging
 1 file changed, 1 insertion(+)
[master dadd1b8] important secret
 1 file changed, 1 insertion(+)
 create mode 100644 private.secret
[master 5a5877d] Add doc - step 1
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
[master 5267760] Add doc - step 2
 1 file changed, 1 insertion(+), 1 deletion(-)
[master 5257f1a] Add doc - step 3
 1 file changed, 2 insertions(+)
[master fa7d9f5] Test for feature hello world
 1 file changed, 1 insertion(+)
 create mode 100644 hello.test
[master 773ddba] I forgot a semicolon
 1 file changed, 1 insertion(+), 1 deletion(-)
[ddnguyen@localhost exercise]$
```

baXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

1:39 PM
3/11/2026

Task 1:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph
* 773ddba (HEAD -> master) I forgot a semicolon
* fa7d9f5 Test for feature hello world
* 5257f1a Add doc - step 3
* 5267760 Add doc - step 2
* 5a5877d Add doc - step 1
* dadd1b8 important secret
* 92147ce debugging
* 3b4fd61 Really made the thingy done
* e158e59 Finished HW feature
* 6fad267 Helo Volrd feature
* 4dd06a2 (tag: v0.0, origin/master) Initial commit
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Task 2:

```
[ddnguyen@localhost exercise]$ git rebase --interactive v0.0
Successfully rebased and updated refs/heads/master.
[ddnguyen@localhost exercise]$
```

Task 3:

```
[ddnguyen@localhost exercise]$ git log --oneline --graph
* 8417dc5 (HEAD -> master) I forgot a semicolon
* 2035f2e Test for feature hello world
* d277499 Add doc - step 3
* df8e17a Add doc - step 2
* b721507 Add doc - step 1
* 4fa0163 important secret
* 0005915 debugging
* 1b4cd63 Really made the thingy done
* 6105e3d Finished HW feature
* 616d655 Helo Volrd feature
* 4dd06a2 (tag: v0.0, origin/master) Initial commit
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

2.14 Rebase using autosquash

```
[ddnguyen@localhost git-katas]$ cat rebase-interactive-autosquash
cat: rebase-interactive-autosquash: Is a directory
[ddnguyen@localhost git-katas]$ cd rebase-interactive-autosquash
[ddnguyen@localhost rebase-interactive-autosquash]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost rebase-interactive-autosquash]$ cat README.md
# Git katas: Interactive rebase with --autosquash option
You have worked on a new feature called Hello World.
This features ends up being complete with both documentation and unit test, but there is a typo in the documentation.

You need to fix it and then rebase to have a beautiful history.

Luckily we have a release tag `v0.0` from just before we started the feature.

There is a way to easily fix it with advanced options for `git commit` and `git rebase`.

## Setup:

1. Run `. setup.sh` (or `.\setup.ps1` in PowerShell)

## Task

1. Explore the repo and the history so you know when the documentation file was added.
2. Fix `README.md` file and add it.
3. Add your commit by using `git commit --fixup=<commit id to be fixed>`.
4. Use `git rebase --autosquash --interactive v0.0` to view the rebase recipe automatically generated.
5. Use `git log` to view your new beautiful history.

### useful commands

- `ls -l`                          # list files
- `tail -n +1 *`                   # show content of all files
- `git log --oneline`              # show history
- `git log --stat`                 # log which files changed
- `git log --patch`                # log with diff
- `git show <commit id>`           # show changes of a commit
- `git add`                        # add file
- `git commit --fixup=<commit id>` # commit by autogenerating the message
- `git rebase -i <ref>`            # run the interactive rebase back to <ref> and automaticaly reorder commits
[ddnguyen@localhost rebase-interactive-autosquash]$
```

Setup:

```
[ddnguyen@localhost rebase-interactive-autosquash]$ . setup.sh
Initialized empty Git repository in /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/rebase-interactive-autosquash/exercise/.git/
[master (root-commit) e1e70cf] Initial commit
[master 0a7f5c3] Hello World feature
 1 file changed, 1 insertion(+)
 create mode 100644 hello.code
[master 82044a8] Add documentation
 1 file changed, 3 insertions(+)
 create mode 100644 README.md
[master 8a7be43] Add Unrelated stuff feature
 1 file changed, 1 insertion(+)
 create mode 100644 unrelated.stuff.code
[master 4f20b48] Test for feature hello world
 1 file changed, 1 insertion(+)
 create mode 100644 hello.test
[ddnguyen@localhost exercise]$
```

baXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

1:48 PM
3/11/2026

Task 1:

```
[ddnguyen@localhost exercise]$ git log
commit 4f20b48259b98b4031241fe737895503191dd321 (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Test for feature hello world

commit 8a7be433acd626208250ec9ec8514678f2b1df42
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Add Unrelated stuff feature

commit 82044a8d639a7a8bd4ec4637550b69c3b2693a0d
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Add documentation

commit 0a7f5c3762b5b96f45ee775f9614f06dc948b413
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Hello World feature

commit e1e70cf6e25c246dc630dec58ec8d54ab80bb346 (tag: v0.0)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Initial commit
```

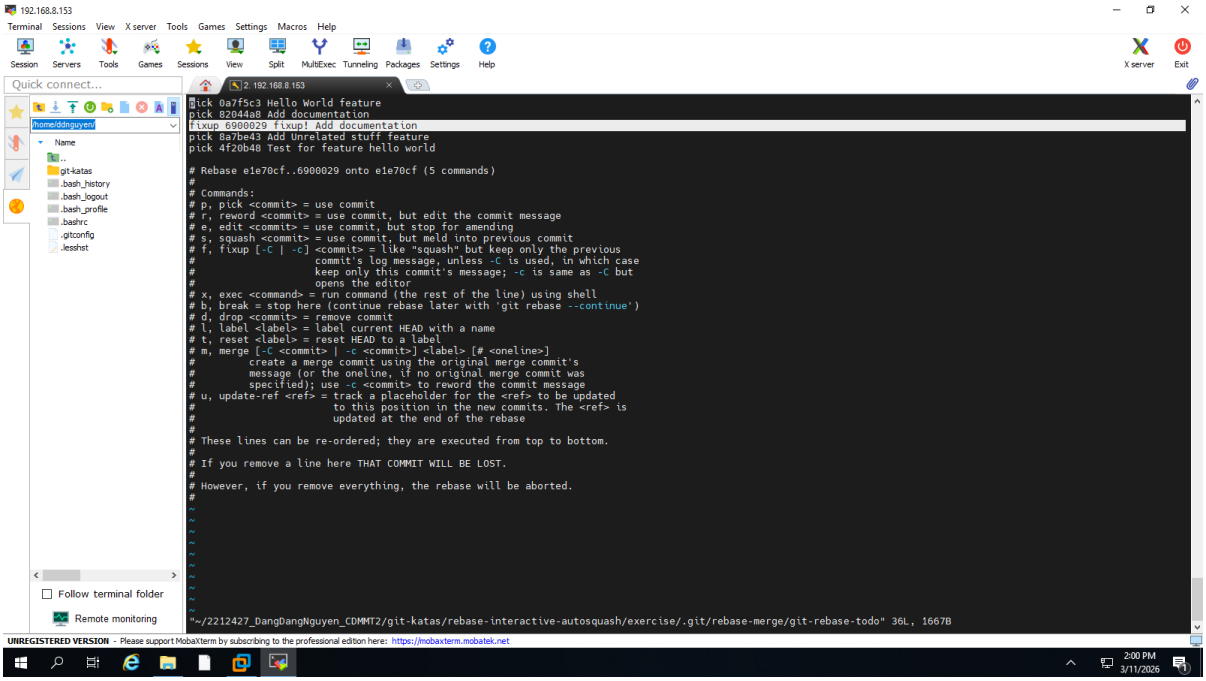
Task 2:

```
This program does exactly what it says
[ddnguyen@localhost exercise]$ nano  README.md
[ddnguyen@localhost exercise]$ git add README.md
[ddnguyen@localhost exercise]$
```

Task 3:

```
[ddnguyen@localhost exercise]$ git commit --fixup=82044a8
[master 6900029] fixup! Add documentation
 1 file changed, 1 insertion(+), 1 deletion(-)
[ddnguyen@localhost exercise]$
```

Task 4:



Task 5:

```
[ddnguyen@localhost exercise]$ git log
commit 7d802f79acb58ed13542dd6e4674bcf2bb1edc62 (HEAD -> master)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Test for feature hello world

commit bbd4d398a480a978ea3b13cbc93fc5c18f07096d
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Add Unrelated stuff feature

commit 482f627881bcfa1ecc037b3ee1cf53c4168dc55c
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Add documentation

commit 0a7f5c3762b5b96f45ee775f9614f06dc948b413
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Hello World feature

commit e1e70cf6e25c246dc630dec58ec8d54ab80bb346 (tag: v0.0)
Author: dang <2212427@dlu.edu.vn>
Date:   Wed Mar 11 13:48:39 2026 +0700

    Initial commit
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

2:00 PM
3/11/2026

2.15 Basic stashing

```
[ddnguyen@localhost git-katas]$ cd basic-stashing
[ddnguyen@localhost basic-stashing]$ ls
README.md  setup.ps1  setup.sh
[ddnguyen@localhost basic-stashing]$ cat README.md
# Git Kata: Basic stashing

## Setup:

1. Run `source setup.sh` (or `.\setup.ps1` in PowerShell)

## The task

You are working on your project. You've staged some work and have some unstaged work as well.
Suddenly, you're made aware that a bug has made it to production. You'll stash your work, fix the bug and get back to your original work.

1. Explore the repo
   1. What work do you have in the working directory?
   2. What work do you have staged ?
   3. What does the commit log look like ?
   >*Notice that file.txt has some staged changes (i.e. changes in the index) and unstaged changes (changes in the working directory)*
2. Use `git stash` to stash your current work.
   1. Now, what work do you have in the working directory?
   2. What work do you have staged ?
   3. What does the commit log look like ?
   4. What does the stash list look like ?
3. Fix the typos in bug.txt on master and commit your changes.
4. Now to get back to your work, apply the stash to master.
   1. What work do you have in the working directory?
   2. What work do you have staged ?
   >*Oops. All our changes are unstaged now. This may be undesirable and unexpected*
5. Undo our changes with `git reset --hard HEAD`. This is an unsafe command as it will remove files from your index and working directory permanently, but we have our
 changes safely stashed so we're ok. Review the [reset](reset/README.md) kata if you're unsure of what happens here.
6. Apply the stash to master with the `--index` option.
   1. What work do you have in the working directory?
   2. What work do you have staged ?
   >*Ok, back to where we were!*
7. We won't need the stash anymore. Drop it.
   1. What does the stash list look like ?
   2. What does the commit log look like ?
```

oaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Setup:

```
[ddnguyen@localhost basic-stashing]$ source setup.sh
Initialized empty Git repository in /home/ddnguyen/2212427_DangDangNguyen_CDMMT2/git-katas/basic-stashing/exercise/.git/
Successfully configured local username and email
Successfully disable local GPG signing
[master (root-commit) ca8e98c] Initial commit
 2 files changed, 1 insertion(+)
 create mode 100644 file.txt
 create mode 100644 fix.txt
[master 693cad7] add bug.txt
 1 file changed, 1 insertion(+)
 create mode 100644 bug.txt
Successfully unset local username and email
Successfully unset local GPG signing
[ddnguyen@localhost exercise]$
```

Task:

```
[ddnguyen@localhost exercise]$ git stash
Saved working directory and index state WIP on master: 693cad7 add bug.txt
[ddnguyen@localhost exercise]$
```

oaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

```
[ddnguyen@localhost exercise]$ git stash list
stash@{0}: WIP on master: 693cad7 add bug.txt
[ddnguyen@localhost exercise]$
```

```
[ddnguyen@localhost exercise]$ nano bug.txt
[ddnguyen@localhost exercise]$
```

obaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

```
[ddnguyen@localhost exercise]$ git add bug.txt
git commit -m "Fix typo in bug.txt"
[master cde377e] Fix typo in bug.txt
 1 file changed, 1 insertion(+)
[ddnguyen@localhost exercise]$
```

MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

```
[ddnguyen@localhost exercise]$ git reset --hard HEAD
HEAD is now at cde377e Fix typo in bug.txt
[ddnguyen@localhost exercise]$
```

```
git commit -m "Fix typo in bug.txt"
[master cde377e] Fix typo in bug.txt
 1 file changed, 1 insertion(+)
[ddnguyen@localhost exercise]$ git reset --hard HEAD
HEAD is now at cde377e Fix typo in bug.txt
[ddnguyen@localhost exercise]$ git stash apply --index
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   file.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt
        modified:   fix.txt
```

```
[ddnguyen@localhost exercise]$ git stash drop
Dropped refs/stash@{0} (f5e4ba92a2b8742bb7709937e303ae7b3e672095)
[ddnguyen@localhost exercise]$
```